

8. Delivery Plan

8.1 Development. Follows the three phases from 7-mvp-post-mvp.md 7.1, with one adjustment: Phase 2 has a real external dependency on Trading/Orders confirming the `order-commands` topic and `OrderRejected` reason-code shape (5.1 #1-#2). Rather than waiting, Phase 2 development begins against a stubbed Trading/Orders consumer/producer (a local fake accepting commands and emitting configurable fill/reject events), swapping to the real integration once confirmed — keeps Phase 2 moving in parallel with that cross-team conversation.

8.2 Testing. Unit tests on the Plan/Execution/Order state machines (1.9–1.10), including transitions explicitly *not* allowed (e.g. `Canceled` never re-entering `Active`). Integration tests for the Outbox → Debezium → Kafka → Projection round trip, and the Dispatch → `execution-triggers` → Initiate fan-out (1.5–1.6). Contract tests against Trading/Orders’ published event/command shapes — turns the unconfirmed assumptions in 5.1 #1-#2 into “verified, alerts on drift.” Load/scale tests targeting the Dispatch/Initiate fan-out (2.4) at the peak-day volume from 6-non-functional-requirements.md 6.1, not just the average. Failure-injection tests: kill the Dispatcher mid-scan and confirm crash recovery (1.5/4.1); redeliver duplicate `OrderFilled/OrderRejected` and confirm `UpdateOrderStatus` is a no-op on an already-terminal Order (4.3). Idempotency tests: duplicate `InitiateExecution` calls for the same (`plan_id`, `execution_month`) resolve to exactly one Execution (2.5).

8.3 Rollout. This moves users’ money on an unattended schedule, so rollout is staged: (1) internal/employee cohort first, against the full pipeline including live Trading/Orders; (2) gate by recurrence day, not user percentage — accept plan creation only for a small number of near-term recurrence days initially, using the dashboards/alerts from 6-non-functional-requirements.md 6.4 as the go/no-go signal before opening the rest; (3) general availability once several consecutive execution days have run clean (dispatch within the alerting cutoff, no rejection-rate spike, no consumer-lag incidents). Rollback doesn’t mean “undo” — every write path is durable and every retry path is idempotent (2.5, 4.3), so rollback means *stop starting new work*: disable plan creation via a feature flag and/or pause the Scheduler. In-flight executions drain naturally through the existing resilient consumers; nothing needs manual reconciliation.